Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study N-gram model	
Experiment No: 7	**Date:**	**Enrolment No: 92301733050**

Aim: To study N-gram model

IDE: Google Colab **Theory:**

Given a sequence of N-1 words, an N-gram model predicts the most probable word that might follow this sequence. It's a probabilistic model that's trained on a corpus of text. Such a model is useful in many NLP applications including speech recognition, machine translation and predictive text input.

An N-gram model is built by counting how often word sequences occur in corpus text and then estimating the probabilities. Since a simple N-gram model has limitations, improvements are often made via smoothing, interpolation and backoff. An N-gram model is one type of a Language Model (LM), which is about finding the probability distribution over word sequences.

Consider two sentences: "There was heavy rain" vs. "There was heavy flood". From experience, we know that the former sentence sounds better. An N-gram model will tell us that "heavy rain" occurs much more often than "heavy flood" in the training corpus. Thus, the first sentence is more probable and will be selected by the model.

A model that simply relies on how often a word occurs without looking at previous words is called unigram. If a model considers only the previous word to predict the current word, then it's called bigram. If two previous words are considered, then it's a trigram model.

An n-gram model for the above example would calculate the following probability:

P('There was heavy rain') = P('There', 'was', ‘heavy', 'rain') = P('There')P('was'|'There')P(‘heavy'|'There was')P(‘rain'|'There was heavy’)

Since it's impractical to calculate these conditional probabilities, using Markov assumption, we approximate this to a bigram model: P('There was heavy rain’) ~ P('There')P('was'|'There')P(‘heavy'|'was')P('rain'|'heavy’) In speech recognition, input may be noisy and this can lead to wrong speech-to-text conversions.

N-gram models can correct this based on their knowledge of the probabilities. Likewise, N-gram models are used in machine translation to produce more natural sentences in the target language.

When correcting for spelling errors, sometimes dictionary lookups will not help. For example, in the phrase "in about fifteen mineuts" the word 'minuets' is a valid dictionary word but it's incorrect in this context. N-gram models can correct such errors.

N-gram models are usually at word level. It's also been used at character level to do stemming, that is, separate the root word from the suffix. By looking at N-gram statistics, we could also classify languages or differentiate

between US and UK spellings. For example, 'sz' is common in Czech; 'gb' and 'kp' are common in Igbo. In general, many NLP applications benefit from N-gram models including part-of-speech tagging, natural language generation, word similarity, sentiment extraction and predictive text input.

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Program (Code):

To be attached with

1. N-Gram Model Code for generating dynamic N-Grams

```
import nltk from nltk.util import ngrams from collections

import defaultdict, Counter import random



# Sample corpus text = """there was heavy rain there was heavy flood

there was a storm the rain was heavy and the flood was dangerous"""



# Preprocessing tokens =

text.lower().split()



# Function to build N-gram model def

build_ngram_model(tokens, n):

   model = defaultdict(Counter)

for gram in ngrams(tokens, n):

      prefix = gram[:-1]
      word = gram[-1]

model[prefix][word] += 1     return

model
```

```
# Function to convert counts to probabilities def

convert_to_prob(model):

   prob_model = {}

for prefix in model:

      total_count = float(sum(model[prefix].values()))        prob_model[prefix] = {word:

count/total_count for word, count in model[prefix].items()}     return prob_model



# Function to predict next word def

predict_next(prob_model, seed):

   seed = tuple(seed)     if

seed in prob_model:

      words = list(prob_model[seed].keys())

probs = list(prob_model[seed].values())

return random.choices(words, probs)[0]

   else:

      return "No prediction available"



# Choose N

N = 3  # Change to 2 (bigram), 3 (trigram), etc.



# Build model



# Test prediction seed = ("there", "was")  # for trigram

print("Next word prediction:", predict_next(prob_model, seed))
```

Results:

To be attached with

```
Next word prediction: a
```

Observation:

- The N-gram model predicts the next word based on previous words using probability.
- Trigram model gives better contextual predictions than unigram and bigram.
- The model correctly predicts **“heavy”** after “there was” due to higher frequency in corpus.
- Accuracy depends heavily on training data size and quality.
- The model fails when unseen word sequences are given (data sparsity problem).
- This limitation can be improved using smoothing techniques.

Post Lab Exercise:

Apply the probability distribution concept to predict the next word by taking sample document and seed phrases. Attach the code and screenshot. Write your observation here.

```
import nltk
from nltk.util import ngrams from
collections import defaultdict, Counter
import random

# Sample document
text = """there was heavy rain there was heavy flood there was a storm the
rain was heavy and the flood was dangerous"""

# Preprocessing
tokens = text.lower().split()
```

```
# Build trigram model
def build_model(tokens, n):
    model = defaultdict(Counter)
    for gram in ngrams(tokens, n):
        prefix = gram[:-1]
        word = gram[-1]
        model[prefix][word] += 1
    return model

# Convert counts to probabilities
def get_probabilities(model):
    prob_model = {}
    for prefix in model:
        total = sum(model[prefix].values())
        prob_model[prefix] = {word: count/total for word, count in model[prefix].items()}
    return prob_model

# Predict next word
def predict_next(prob_model, seed):
    seed = tuple(seed)
    if seed in prob_model:
        words = list(prob_model[seed].keys())
        probs = list(prob_model[seed].values())
        return random.choices(words, probs)[0]
    else:
        return "No prediction"

# Generate sentence
def generate_text(prob_model, seed, n_words):
    result = list(seed)
    for _ in range(n_words):
        next_word = predict_next(prob_model, seed)
        if next_word == "No prediction":
            break
        result.append(next_word)
        seed = seed[1:] + [next_word]
    return " ".join(result)

# Build and test
model = build_model(tokens, 3)
prob_model = get_probabilities(model)

seed = ["there", "was"]
print("Next word:", predict_next(prob_model, seed))
print("Generated sentence:", generate_text(prob_model, seed, 5))
```

Output:

```
Next word: a
Generated sentence: there was heavy rain there was heavy
```

Observations:

- The probability distribution helps in selecting the most likely next word based on previous words.
- The model predicted **"heavy"** after the phrase "there was" because it appeared frequently in the dataset.
- Trigram model gives more meaningful predictions compared to unigram and bigram.
- The generated sentence follows the learned pattern from the training text.
- If the input seed phrase is not present in the training data, the model fails to predict (data sparsity issue).
- The performance improves with larger datasets and smoothing techniques.